Lab 3 (Backtracking in Prolog, CUT, FAIL)

What is Backtracking?

Backtracking is the process Prolog uses to:

- Try a possible solution
- If it fails, go back to the previous choice point
- Try the next available option

"**Backtracking**" is called so because Prolog:

- **Backs up** to previous steps when a path **fails**
- **Tries a different route** until a solution is found or all paths fail

Example: Choosing a Travel Route

```
% --- Travel connections ---
path(a, b).
path(b, c).
path(a, d).
path(d, c).
path(a, e).

% --- Rule: Can travel from X to Y via two steps
can_travel(X, Y) :-
        path(X, Z),
        path(Z, Y).
```

Query to Try:

?- `can_travel(a, c).`

Step-by-Step

First, Prolog tries:

```
path(a, b), path(b, c).
```

Success! So it returns: true.

Suppose we remove the first working route:

```
% Remove path(b, c).
```

Now facts are:

```
path(a, b).
path(a, d).
path(d, c).
path(a, e).
```

Now try the same query again:

?- `can_travel(a, c).`

```
% path(b, c). ← removed
```

Backtracking In Action

Tries: path(a, b) (succeed)
Tries: path(b, c) (failed)

Backtracks:

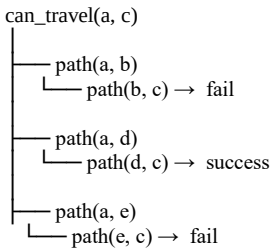
Tries next: path(a, d) (Succeed)
Tries: path(d, c) (succeed)

Succeeds through alternate path: a → d → c

Output:

true.

Tree diagram:



Because Prolog uses backtracking to explore all possible matches for path(a, Z). So once it tries:

path(b, c) → fails does not exist)
path(d, c) → succeeds (exists)
Even though a success is found, if you press; in the Prolog shell, it continues looking.
Then it tries path(e, c) → does not exist

So even though the first path failed, Prolog backtracked to the next possibility (path(a, d)) and found a working solution.

Why This is True Backtracking

- First route failed
- Prolog backtracked to try `a → d → c`
- Found success through a different path

This demonstrates:

"**Backtracking after failure** to try other solutions"

- **Is Backtracking Still Happening If Nothing Fails?**

Yes — even if none of the choices actually fail, backtracking is still involved behind the scenes because Prolog is exploring multiple solutions, one at a time.

Concept	What It Means
Failure-triggered backtracking	Happens when Prolog hits a dead end (e.g., wrong fact, no match)
Choice-point backtracking	Happens when Prolog knows there are alternatives, and you ask it to try them

Let us consider the example:

```
color(apple, red).
color(apple, green).
color(apple, yellow).
```

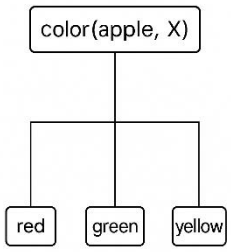
Query

```
?- color(apple, X).
```

You're asking Prolog:

"What are the values of `X` such that `color(apple, X)` is true?"

Step by step execution:



- Prolog starts at the root of the tree: color(apple, X)
- It explores the first branch: X = red
- If the user presses ;, Prolog backtracks to try the next option:

  Goes to X = green

- Pressing **;** again makes Prolog backtrack again:

  Tries X = yellow

- If **;** is pressed once more and there are no other facts, Prolog reaches a dead end and returns false.

Does Prolog always backtrack after success?

No — only if:
You explicitly request more results.
You're in a multi-solution context (e.g., findall, loops, etc.).

When Backtracking is Useful

- Exploring all possible results
- Solving logic puzzles
- Writing search algorithms
- Matching multiple patterns

Assignment Title:

Implementing a Route-Finding Search Algorithm Using Backtracking in Prolog

Objective:

To implement a depth-first search (DFS)-style route-finding algorithm using Prolog's natural backtracking mechanism. Students will model a map of cities and find all possible paths from a source to a destination.

Task Description:

Define a set of facts representing direct routes between cities:

```
route(a, b).
route(a, c).
route(b, d).
route(c, d).
route(d, e).
```

Write a Prolog predicate path(Source, Destination, PathList) which:

- Finds a path from Source to Destination.
- Returns the sequence of cities traversed as PathList.
- Uses recursive rules and backtracking to explore all paths.
- Add a version to find all possible paths using findall/3.

- **Advanced:** Extend the program with cycle detection using an accumulator.

Sample Program:

```
% --- Direct Routes ---
route(a, b).
route(a, c).
route(b, d).
route(c, d).
route(d, e).

% --- Base Rule: Direct path ---
path(X, Y, [X, Y]) :-
    route(X, Y).

% --- Recursive Rule: Path through Z ---
path(X, Y, [X | Rest]) :-
    route(X, Z),
    path(Z, Y, Rest).

% --- Collect all paths using findall ---
all_paths(Source, Dest, AllPaths) :-
    findall(Path, path(Source, Dest, Path), AllPaths).
```

Example Base Case Query:

```
?- path(d, e, Path).
```

Why this triggers the base case:

We have the fact:
route(d, e).
So Prolog will match:
path(d, e, [d, e]) :- route(d, e).
It succeeds immediately, without recursion.

Output:
Path = [d, e].

Recursive Rule: Path through Z ---

```
?- path(a, e, Path)
```

Try recursive rule (no direct route from a → e):
route(a, Z) gives:

Z = b
Z = c

Try: Z = b

path(b, e, Rest)
route(b, d) ⇒ Try: path(d, e, Rest)
route(d, e) ⇒ base case fires ⇒ path(d, e, [d, e])

Backtrack:
Rest = [d, e]
Rest = [b, d, e]
Path = [a, b, d, e]

Backtrack:

Try Z = c
path(c, e, Rest)
route(c, d) ⇒ Try: path(d, e, Rest)
route(d, e) ⇒ base case fires ⇒ path(d, e, [d, e])

Backtrack:

Rest = [d, e]
Rest = [c, d, e]
Path = [a, c, d, e]

% --- Collect all paths using findall ---

Prolog sees:

all_paths(a, e, AllPaths) :- findall(Path, path(a, e, Path), AllPaths).

Now Prolog starts solving path(a, e, Path) for each possibility:

First Attempt:

route(a, b) → try path(b, e, Rest)
route(b, d) → try path(d, e, Rest)
route(d, e) → base case: path(d, e, [d, e])
build back:
path(b, e, [b, d, e])
path(a, e, [a, b, d, e])
One solution found: [a, b, d, e]

Backtrack to try another option:

backtrack to route(a, c)
route(a, c) → try path(c, e, Rest)
route(c, d) → try path(d, e, Rest)
route(d, e) → base case: path(d, e, [d, e])
build back:
path(c, e, [c, d, e])
path(a, e, [a, c, d, e])
Second solution: [a, c, d, e]

findall/3 collects both paths:

AllPaths = [[a, b, d, e], [a, c, d, e]]

Final Result:

?- all_paths(a, e, AllPaths).
AllPaths = [[a, b, d, e], [a, c, d, e]].

Assignment Title:
Solving Logic Puzzles Using Backtracking in Prolog

Objective:

To implement a Prolog program that solves a classic logic puzzle by systematically searching and backtracking through possible solutions until a consistent one is found.

Puzzle Description:

The Zebra Puzzle (Simplified Version)
There are three houses in a row, each painted a different color (red, blue, green)
Each house is inhabited by a person with a different pet (dog, cat, bird).
Use the clues below to determine who owns the cat.

Clues:

The red house is to the left of the green house
The person in the red house owns the dog
The blue house is in the middle.
The person who owns the bird lives next to the person with the cat.

Tasks:

Represent houses, colors, and pets as Prolog facts and structures
Define constraints from the clues as rules.
Use backtracking to explore valid combinations.
Use trace to understand how backtracking works.
Output the owner of the cat.